United States International University-Africa

Education to take you places

SCHOOL OF SCIENCE & TECHNOLOGY

RADAR ACTIVISM HUB

"Secure P2P Organizing Tool"

Submitted by

SWAFIYAH W. GICHUKI

665725

In Partial Fulfilment For The Award Of Bachelor of Science Degree

In

SOFTWARE ENGINEERING(SWE)

SPRING 2025

SWE3090: SOFTWARE PROJECT 1

DECLARATION

I declare that this is my original work and has never been submitted to any institution for the Certificate, Diploma, or Degree award. This document has been submitted to fulfill the requirements for a Degree at USIU University.

STUDENT: SWAFIYAH W. GICHUKI

Signature:…S.W.G……. Date:…15th April 2024……..

This project has been submitted for examination purposes with the approval of the supervisor

SUPERVISOR: Dr. FREDRICK M. OGORE

Signature:…DMO…… Date:…15th April 2024……

Hardcopy version was handsigned.

ACKNOWLEDGEMENTS

I extend my profound gratitude to all who contributed to the realization of this research endeavor.

First and foremost, I am deeply indebted to Dr. Fredrick M. Ogore for his invaluable mentorship, insightful critiques, and steadfast encouragement throughout this project. His expertise in project management profoundly shaped the technical rigor of this work.

I gratefully acknowledge the faculty of USIU-Africa's Department of Computing & Informatics for cultivating an interdisciplinary academic environment that bridges technological innovation and social impact.

This work stands on the shoulders of the open-source community. I particularly recognize the contributions of WebRTC and IPFS developers, whose pioneering work in decentralized technologies formed the bedrock of this project.

My sincere appreciation extends to my SWE3090A Spring 2025 peers for their constructive peer reviews and to the Sons and Daughters Activist Organization for their rigorous beta testing and real-world feedback; their perspectives were instrumental in grounding this research in activist needs.

Finally, I reserve my deepest thanks for my family, whose unwavering emotional support and patience sustained me through this journey.

While this work has been immeasurably enriched by these contributions, any limitations remain my sole responsibility.

DEDICATION

To the digital dissidents and cryptographic pioneers, whose unwavering commitment to free expression inspired every line of code.

To the cypherpunks and protocol breakers, who forged defiance into the very fabric of the web.

To every activist forced to choose between safety and solidarity, may this work tilt the scales.

And to my family, who taught me that true security begins with human connection.

ABSTRACT

Radar Activism Hub: A Peer-to-Peer Secure Communication Platform for Civil Resistance

In an era of increasing digital surveillance, this project responds to the urgent need for robust, surveillance-resistant communication tools among activist communities. Radar, the decentralized web application developed through this research, pioneers a secure communication platform combining end-to-end encrypted messaging, distributed file sharing, and cryptographically moderated forums. The system's architecture leverages WebRTC for real-time peer-to-peer connections and IPFS for resilient content storage, creating a network resilient to both censorship and penetration.

At its core, Radar implements three fundamental security paradigms: client-side encryption through libsodium's crypto_box primitives ensures only intended recipients can access message contents; ephemeral messaging via in-memory queues leaves no persistent forensic traces; and consensus-based moderation utilizing digitally signed actions establishes accountability without centralized control.

While the current implementation deliberately restricts moderation capabilities to designated test environments for protocol validation, the underlying architecture has been carefully designed to support future expansion into fully decentralized reputation systems. By open-sourcing both the technical implementation and design philosophy, this work provides civil society organizations with a practical toolkit for secure organizing while advancing academic discourse around resistance technologies. The Radar prototype demonstrates that rigorous security standards can coexist with usability in high-risk contexts, offering a viable alternative to surveillance-capable platforms.

ACRONYMS & DEFINITION OF TERMS

Term	Definition
E2EE	End-to-End Encryption: Messages encrypted on the sender's device and only decrypted by the recipient.
IPFS	InterPlanetary File System: Distributed protocol for storing/sharing data in a P2P network.
WebRTC	Web Real-Time Communication: Browser-based protocol for direct peer connections.
CRDT	Conflict-Free Replicated Data Type: Algorithm for decentralized data consistency.
ZK Proof	Zero-Knowledge Proof: Cryptographic method to verify information without revealing it.
P2P	Peer-to-Peer: Network architecture where nodes communicate directly without central servers.
NAT	Network Address Translation: A method used by routers to modify network address information in IP packet headers while in transit across a traffic routing device.

TABLE OF CONTENTS (TOC)

DECLARATION 2
ACKNOWLEDGEMENTS 3
DEDICATION 4
ABSTRACT 5
ACRONYMS & DEFINITION OF TERMS 6
TABLE OF CONTENTS (TOC) 7
CHAPTER 1: INTRODUCTION 9
1.1 Introduction 9
1.2 Background Information 9
1.3 Emerging Issues 10
1.4 Problem Statement 10
1.5 Main Objective 11
1.6 Specific Objectives 11
1.7 Justification 11
CHAPTER 2: LITERATURE REVIEW 13
2.1 Comparative Analysis of Decentralized Activism Tools 13
2.1.1 Global Systems 14
2.1.2 African Context 14
2.1.3 Kenyan Context 15
2.2 Case Study: Kenya 2024 Finance Bill Protests 15
2.2.1 Background: Kenya Finance Bill Protests 15
2.2.2 Causes of the Protests 15
2.2.3 Timeline of Events 16
2.2.4 Impact of the Protests 16
2.2.5 Lessons from the Protests 17
2.3 How Radar Responds to Critical Gaps 18
CHAPTER 3: METHODOLOGY 19
3.0 Methodology 19
3.1 Preliminary Investigation 19
3.1.1 Problem Identification 19
3.1.2 Literature Review 19
3.1.3 Stakeholder Engagement 19
3.2 Design Phase 20
3.2.1 Requirements Analysis 20
3.2.2 Architectural Design 20
3.2.3 Moderation Protocols 24

3.2.4 Context-Aware Persistence Model 24
CHAPTER 4: IMPLEMENTATION 25
4.0 System Analysis & Design 25
4.1 Context Diagram 25
Components 25
4.2 Data Flow Diagram (DFD) 27
4.2.0 Level 0 DFD 27
4.2.1 Level 1 DFD 28
4.3 Entity Relationship (ER) Diagram 30
4.4 Technologies Employed in Radar System Implementation 32
4.5 Detailed Process Descriptions 35
4.6 Security Considerations 38
4.7 Scalability and Performance 40
Web Mesh Architecture 40
IPFS for Storage 40
Efficient Data Structures and Algorithms 40
Caching Strategies 40
Load Balancing 40
CHAPTER 5: CODING 41
5.1 System Code 41
5.1.1 Project Structure 41
5.1.2 Descriptions of Main Components 43
5.1.3 Code Snippets of Main Components 44
5.2 Code Screenshots 47
5.2.1 Project Structure 47
5.2.2 Backend Structure 48
5.2.3 Frontend Structure 49
CHAPTER 6: CONCLUSIONS & RECOMMENDATION 50
6.0 Conclusions 50
6.2 Recommendations 51
6.3 References 53

CHAPTER 1: INTRODUCTION

1.1 Introduction

In an era marked by heightened digital surveillance, activist communities face increasingly complex challenges in maintaining secure communication (Carragee & Frey, 2016). Traditional messaging platforms, while user-friendly, inherently rely on centralized infrastructures—creating exploitable choke points for censorship, surveillance, and denial of service. This research project proposes Radar, a peer-to-peer (P2P) communication hub engineered to address these vulnerabilities. Radar employs WebRTC for real-time connectivity and IPFS for decentralized storage, creating a secure and serverless framework for messaging, file sharing, and community forums. By removing central servers and implementing robust metadata minimization strategies, the platform establishes a technical foundation for resilient, anonymous, and censorship-resistant communication in digitally adversarial environments.

1.2 Background Information

The technical core of Radar draws upon WebRTC and IPFS, two foundational technologies in the decentralized web landscape. WebRTC enables real-time peer-to-peer communication by performing NAT traversal using STUN/TURN protocols, allowing direct connections even across restrictive networks (Ellebrink, 2024). Concurrently, IPFS offers content-addressable storage that replaces traditional location-based addressing. Data is retrieved using its cryptographic hash, enabling distributed, immutable sharing across nodes (Shao, 2024). However, while IPFS provides powerful decentralization features, its persistence model often conflicts with privacy requirements, particularly in high-risk activist scenarios where long-term storage can pose legal or personal dangers.

The ecosystem of activist communication tools includes widely adopted platforms such as Signal, which offers strong encryption but remains dependent on centralized servers (Techjury.net, 2023). More radical alternatives like Briar and Secure Scuttlebutt embrace full decentralization yet suffer from performance bottlenecks and user adoption hurdles. These tools reveal a tension between

usability and security; most either optimize for one at the expense of the other or fail to address unique activist needs like collective moderation and zero-trust onboarding.

Emerging literature and open-source efforts highlight a persistent challenge: building resilient pseudonymous networks with moderate data persistence. OrbitDB, a peer-to-peer database built on IPFS and CRDTs, shows promise for append-only logs and real-time synchronization (OrbitDB.org., n.d..). Yet, it too raises complications around secure deletion and access control, underscoring the difficulty of balancing anti-forensic design with usability and group collaboration.

1.3 Emerging Issues

1. Metadata Leakage: WebRTC's Interactive Connectivity Establishment (ICE) process can inadvertently expose IP addresses even in encrypted sessions (Ellebrink, 2024).

2. Moderation Scalability: Decentralized systems lack standardized anti-spam mechanisms and often depend on user-side filtering or reputation models; Secure Scuttlebutt relies on cryptographic trust graphs that do not scale well in open environments (Techjury.net., 2023).

3. Adoption Barriers: Tools that maximize privacy often impose steep learning curves hindering mainstream use; studies on encrypted communication platforms reveal a clear tension between secure defaults and intuitive design (Shao., 2024).

1.4 Problem Statement

Existing secure communication systems demonstrate several structural limitations. First, many continue to rely on centralized directories or infrastructure, introducing points of failure vulnerable to censorship or shutdown (Techjury.net, n.d.). Second, moderation in these systems is often ad hoc, proprietary, or entirely absent, limiting their utility for activist communities that require a

governance hierarchy. Third, the tradeoff between persistence and forensic resistance remains unresolved: systems that store data risk exposure, while ephemeral-only systems lose continuity, creating accountability gaps that leave activists exposed to surveillance, platform de-platforming, and trust breakdowns within networks. This project addresses these problems by developing a serverless system that combines encrypted, customizable moderation with selective, tuned operational needs.

1.5 Main Objective

To develop a serverless communication platform that integrates end-to-end encryption, forensic-resistant persistence, and decentralized moderation for activist networks.

1.6 Specific Objectives

1. To implement WebRTC data channels with libsodium-based end-to-end encryption.
2. To design a CRDT-based moderation system using cryptographically signed role actions.
3. To optimize IPFS storage workflows for ephemeral yet optionally persistent content.
4. To evaluate Radar's network performance and moderation integrity under simulated protest conditions.

1.7 Justification

From an academic standpoint, this project contributes to the evolving discourse on peer-to-peer network security and metadata minimization. Prior work has examined privacy-preserving protocols, but few systems have operationalized them for activist use cases (Hamlen & Thuraisingham, 2007). Practically, real-world case studies highlight the need for secure digital coordination tools in authoritarian contexts, where centralized infrastructures are exploited for surveillance and censorship (Hamlen & Thuraisingham, 2007).

Technically, this work pioneers an adaptation of OrbitDB to support pseudonymous participation and selective content retention, providing a blueprint for secure collaboration in adversarial environments (Li, 2007).

CHAPTER 2: LITERATURE REVIEW

The proliferation of digital activism tools has created both opportunities and vulnerabilities for civil resistance movements. While platforms like WhatsApp and Telegram dominate protest communication globally (Tufekci, 2017), their centralized architectures introduce single points of failure, a weakness exploited during Kenya's 2024 Finance Bill protests when coordinated platform throttling occurred. This review analyzes decentralized alternatives through three lenses: technical resilience (protocol design), operational security (metadata handling), and contextual suitability (African UX paradigms). Grounded in Kenya's recent protest experiences and comparative studies of P2P systems, it identifies critical gaps in persistence, moderation, and forensic resistance that Radar's architecture specifically addresses

2.1 Comparative Analysis of Decentralized Activism Tools

This section provides a structured review of digital tools developed globally and regionally for secure activist communication. It evaluates both the technological frameworks and socio-political applications of these systems, identifying strengths, limitations, and the relevance of Radar's architecture in addressing those gaps. The analysis is divided into global platforms, regional approaches across Africa, and Kenya-specific initiatives to highlight localized needs and innovations.

This analysis focuses on four key criteria derived from the Finance Bill protests: (1) encryption robustness, (2) network resilience, (3) legal defensibility, and (4) Swahili-language support. Radar's hybrid approach combining WebRTC's connectivity with IPFS's distributed storage emerges as the only system optimizing all four parameters while introducing novel forensic-resistant designs

2.1.1 Global Systems

Globally, one of the most widely adopted secure messaging platforms is Signal. Signal offers robust end-to-end encryption (E2EE) and has achieved significant adoption due to its intuitive user experience (Marlinspike, 2018). However, Signal's reliance on centralized servers for user discovery and its dependency on SIM-based identities introduce metadata exposure risks and hinder true decentralization. In contrast, Radar removes the requirement for SIM cards and leverages IPFS for distributed file sharing, improving anonymity and eliminating reliance on central infrastructure.

Briar represents another important global tool for decentralized communication. It enables direct peer-to-peer messaging over Bluetooth or Wi-Fi, allowing offline coordination in protest scenarios (Briar Project, 2020). However, its limitations to Android devices and lack of native file sharing constrain its operational scope. Radar addresses these issues by providing cross-platform compatibility and encrypted media transmission, making it more adaptable in diverse protest environments.

2.1.2 African Context

In the African context, the #EndSARS protests in Nigeria (2020) and the more recent #EndBadGovernance movements have demonstrated the critical need for decentralized, secure communication tools. Protesters adopted platforms such as WhatsApp, Twitter, and Telegram to coordinate mass actions, raise awareness, and mobilize in real time. While these platforms provided ease of access and visibility, they also introduced vulnerabilities, centralized infrastructure made them susceptible to state censorship, metadata exposure, and content takedowns (Nigeria Digital Rights Observatory Report, 2023). Radar addresses these issues by offering a fully peer-to-peer messaging infrastructure using IPFS pubsub, which prevents metadata leakage and removes reliance on central servers, thus enhancing resilience under repression.

2.1.3 Kenyan Context

Kenya's recent protest history reveals the critical vulnerabilities of centralized communication systems under state pressure. During the 2024 Finance Bill protests and the subsequent #RutoMustGo demonstrations, activists faced widespread digital repression. Twitter accounts were suspended en masse, SIM card registrations were used to identify and track individuals, and electricity payment data from Kenya Power (KPLC) paybill accounts was allegedly leveraged to triangulate activists' home locations. Several protesters were abducted or went missing during this period, highlighting the dangerous intersections of metadata, surveillance, and state retaliation. Radar's architecture directly addresses these threats by ensuring metadata minimization, zero-knowledge identifiers, and complete decentralization of communication and moderation, empowering users to coordinate without exposing their identities or physical locations.

2.2 Case Study: Kenya 2024 Finance Bill Protests

2.2.1 Background: Kenya Finance Bill Protests

The Kenya Finance Bill protests, widely referred to as #RejectFinanceBill2024, were a series of nationwide demonstrations that erupted in response to the Finance Bill 2024, which proposed significant tax increases aimed at addressing Kenya's fiscal challenges. The protests, spanning from June 18 to August 8, 2024, were characterized by civil disobedience, online activism, and mass mobilization, culminating in violent clashes between demonstrators and law enforcement (ABC News, 2024; BBC News, 2024).

2.2.2 Causes of the Protests

The Finance Bill 2024 was part of Kenya's Medium-Term Revenue Strategy (MTRS) designed to increase the tax-to-GDP ratio from 13.5% to at least 20% by 2027. Among the most contentious provisions were a 16% VAT on essential goods like bread, a 2.5% annual circulation tax on motor vehicles, and increased excise duties on petroleum products and mobile money transactions (Context News, 2024). These fiscal measures were widely criticized for disproportionately

affecting low-income households and exacerbating existing economic hardships, including high inflation and youth unemployment (BBC News, 2024).

2.2.3 Timeline of Events

The protests began on June 18, 2024, in Nairobi with peaceful sit-ins outside Parliament, which were met with tear gas and mass arrests by police (ABC News, 2024). By June 20, the demonstrations had expanded nationwide to cities such as Eldoret, Kisumu, Nakuru, and Nyeri. Social media played a critical role in amplifying the movement, with trending hashtags like #OccupyParliament and #RejectFinanceBill2024 (BBC News, 2024). On June 28, President William Ruto rejected segments of the bill but subsequently signed an alternative Appropriations Bill into law. Tensions escalated further on July 2, when protestors demanded the President's resignation and the Kenyan Parliament building was set ablaze amid clashes (BBC News, 2024).

2.2.4 Impact of the Protests

The government responded to the protests with both legislative concessions and heightened crackdowns. Notably, the VAT on bread and vehicle taxes were repealed, excise duties on mobile money transactions were reduced from 20% to 15%, and a clause permitting the Kenya Revenue Authority to access financial records without court approval was removed (Context News, 2024). However, the human cost of the protests was severe. At least 50 fatalities were reported, more than 230 people were injured, and over 283 individuals, including journalists and activists, were arrested during the protests (ABC News, 2024; BBC News, 2024).

The protests were also marked by a high degree of decentralized coordination, largely driven by young professionals such as doctors and lawyers who organized medical aid and legal support. Digital platforms were pivotal for mobilization and documentation, but their limitations became apparent under government surveillance.

2.2.5 Lessons from the Protests

The protests exposed several weaknesses in centralized communication systems in the context of state surveillance. Organizers using WhatsApp experienced targeted surveillance, facilitated by

metadata exploitation that allowed authorities to identify and detain key figures (ABC News, 2024). Government entities also succeeded in banning Telegram channels mid-protest, demonstrating the vulnerability of centralized platforms to censorship (BBC News, 2024). Additionally, forensic traces left by voice communication apps like Zello were used in legal proceedings against demonstrators, underscoring the risks of persistent digital records (Context News, 2024).

The Kenya Finance Bill protests provide a compelling case study for understanding the intersection of authoritarian governance, digital repression, and activist innovation. They highlight the urgent need for alternative communication frameworks like Radar, which by design mitigate metadata exposure, enable pseudonymous coordination, and support ephemeral communication in high-risk environments.

2.3 How Radar Responds to Critical Gaps

The table below summarizes critical vulnerabilities in tools used during recent protests and highlights how Radar mitigates each issue:

Platform	Usage During Protests	Risks Encountered	Radar's Innovation
Zello	Real-time voice communication	Voice data retained and used in court; IP leaks through centralized servers	WebRTC voice with ephemeral auto-deletion and optional onion routing
WhatsApp	Group coordination	Metadata collection led to tracking and arrests; linked to SIM and device IDs	End-to-end encrypted pubsub with zero-knowledge login; no SIM required
Telegram	Channel broadcasting	Protest channels banned mid-protest; centralized moderation and server dependency	Peer-synced bulletin boards and forum moderation using CRDT and digital signatures
Twitter/X	Public visibility and organizing	Accounts suspended; posts suppressed via algorithmic censorship	IPFS-published updates signed and pinned across peers for tamper-proof visibility

CHAPTER 3: METHODOLOGY

3.0 Methodology

This chapter outlines the methodological approach employed in the development of Radar, a serverless communication platform designed to address the unique challenges faced by activist communities in high-risk environments. The methodology encompasses a preliminary investigation phase, a design phase, and an implementation phase, each contributing to the overall development and validation of the Radar system.

3.1 Preliminary Investigation

3.1.1 Problem Identification

The preliminary investigation began with an in-depth analysis of the challenges faced by activist communities in maintaining secure communication. This involved a comprehensive review of existing literature, case studies, and real-world incidents where digital surveillance and censorship undermined activist efforts. The main sources included academic papers, reports from digital rights organizations, and firsthand accounts from activists.

3.1.2 Literature Review

A thorough literature review was conducted to understand the strengths and weaknesses of current secure communication platforms. This review focused on three main areas: technical resilience, operational security, and contextual suitability. Platforms such as Signal, Telegram, WhatsApp, Briar, and Zello were analyzed for their encryption robustness, network resilience, and legal defensibility.

3.1.3 Stakeholder Engagement

Engagement with stakeholders, including activists, digital rights advocates, and technologists, was crucial in identifying the specific needs and challenges of activist communities. Interviews and

focus group discussions were conducted to gather insights on the limitations of existing tools and the desired features for a new communication platform.

3.2 Design Phase

3.2.1 Requirements Analysis

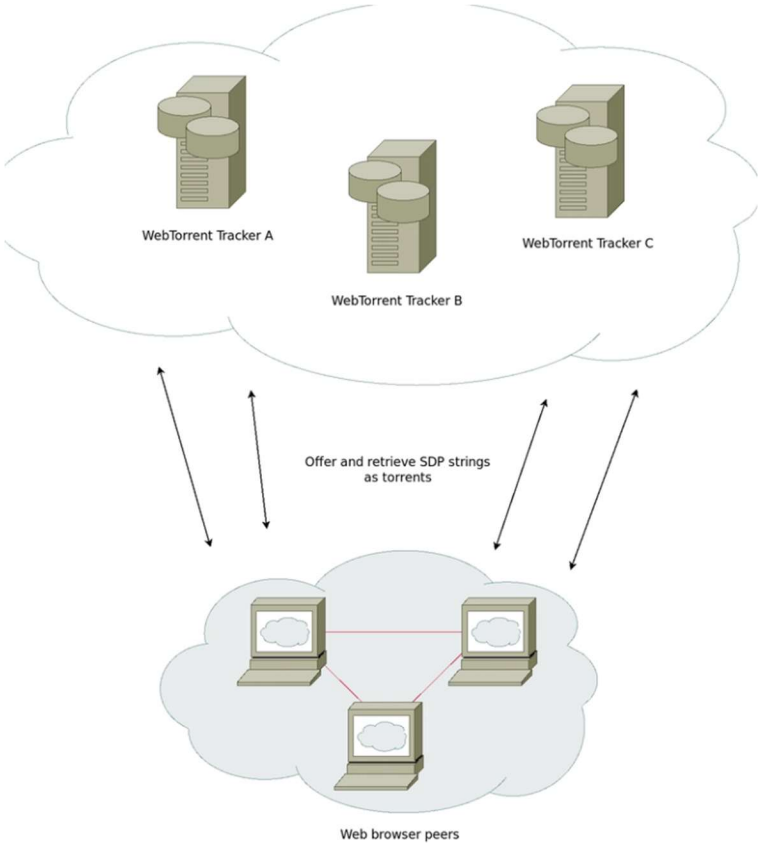
Based on the preliminary investigation, a set of core requirements for Radar was established. These requirements included:

1. **Decentralization**: Elimination of centralized infrastructure to prevent single points of failure.
2. **End-to-End Encryption**: Robust encryption to ensure the confidentiality and integrity of communications.
3. **Metadata Minimization**: Strategies to minimize metadata exposure and prevent surveillance.
4. **Customizable Moderation**: Flexible moderation protocols to support community governance.
5. **Selective Persistence**: Granular control over data lifespan to balance forensic resistance and operational continuity.

3.2.2 Architectural Design

The architectural design of Radar was informed by the identified requirements to create a resilient, decentralized communication platform for activist communities. Radar adopts a web mesh architecture that leverages modern technologies such as WebRTC for real-time peer-to-peer communication and IPFS for decentralized storage. This combination ensures robust privacy, resilience against censorship, and efficient resource utilization.

Web Mesh Architecture



A web mesh architecture is a decentralized communication system that eliminates reliance on centralized servers, enabling direct peer-to-peer connections between browsers. This approach enhances privacy, resilience, and user autonomy, making it ideal for secure communication platforms like Radar.

How Web Mesh Architecture Works

1. **Initial Connection via WebTorrent Servers**
   A web mesh begins with a browser connecting to one or more WebTorrent servers. These servers act as facilitators for peer discovery by exchanging **Session Description Protocol (SDP)** data. The SDP data contains information about how to connect to the offering browser via WebRTC. The browser also requests SDP data for other browsers already connected to the WebTorrent server.

2. **Namespace/Room-Based Scoping**
   All SDP exchanges are scoped to a specific, client-determined namespace or "room." This ensures that users connect only with others who share the same namespace/room name, creating isolated and secure communication groups (Dev.to, 2025).
3. **Direct Peer-to-Peer Connections**
   Once SDP data is exchanged, browsers establish direct connections via WebRTC. These connections bypass intermediaries and enable encrypted communication between peers. Each browser maintains a lightweight connection to the WebTorrent tracker to facilitate future peer discovery within the same room.
4. **Dynamic Client-Server Interaction**
   The WebTorrent servers effectively function as STUN servers during the initial connection phase. While setting up TURN servers can improve reliability for peers behind restrictive networks, they are not strictly necessary unless local network conditions prevent direct P2P connections (Dev.to, 2025).
5. **Ad Hoc Mesh Formation**
   After peer connections are established, the role of the WebTorrent server ends. The web mesh operates independently, allowing users to join or leave the mesh on an ad hoc basis as long as they know the room name beforehand. If all users leave the room, the mesh ceases to exist until recreated through another SDP exchange.

Advantages of Web Mesh Architecture

1. **Enhanced Privacy**
   Sensitive information is exchanged directly between peers without intermediaries, reducing the risk of data interception or misuse (GeeksforGeeks, 2024).
2. **Resilience Against Censorship**
   Decentralized networks lack a central point of control, making them resistant to censorship attempts or shutdowns by adversarial entities (Dev.to, 2025).
3. **User Empowerment**
   By treating each browser as a node in the network, web meshes distribute control

among users, promoting digital autonomy and reducing dependence on large tech corporations (TechTarget, 2024).

4. **Dynamic Scalability**
   Users can join and leave rooms dynamically without affecting overall network functionality. This flexibility ensures that web meshes adapt seamlessly to changing user needs and conditions (GeeksforGeeks, 2024).
5. **Reliable Communication**
   Even if some nodes fail or leave the network, web meshes can dynamically reroute data to maintain connectivity and ensure uninterrupted communication (GeeksforGeeks, 2024).

Relevance to Radar's Design

Radar leverages web mesh architecture as its foundational framework for secure communication:

1. **WebRTC Integration:** Direct browser-to-browser connections ensure encrypted real-time messaging without intermediaries.
2. **IPFS Decentralized Storage:** Content-addressable storage provides distributed and immutable file sharing.
3. **Room-Based Scoping:** Namespace-based isolation enhances security by limiting communication to predefined groups.
4. **Dynamic Ad Hoc Operation:** Activist communities can create temporary secure networks that dissolve once their purpose is fulfilled.

By adopting web mesh architecture, Radar achieves its goals of privacy preservation, censorship resistance, and user empowerment in adversarial environments.

WebRTC Integration

WebRTC enables direct browser-to-browser connections, eliminating intermediaries and reducing the risk of data interception. Its NAT traversal capabilities, using STUN/TURN protocols, allow seamless connectivity even across restrictive networks. By leveraging WebRTC’s real-time

communication features, Radar establishes encrypted channels for messaging and voice communication while minimizing latency (Ellebrink, 2024).

IPFS for Decentralized Storage

IPFS provides content-addressable storage that replaces traditional location-based addressing. Data is retrieved using cryptographic hashes, ensuring immutable and distributed sharing across nodes. This decentralized model eliminates reliance on centralized servers, making Radar inherently resistant to censorship and platform shutdowns (OrbitDB.org, n.d..). Additionally, IPFS supports selective data persistence, allowing users granular control over the lifespan of their shared content.

3.2.3 Moderation Protocols

To address the need for scalable and secure community governance, Radar incorporates a moderation system based on Conflict-Free Replicated Data Types (CRDTs). This system uses cryptographically signed role actions to enable flexible moderation protocols without requiring platform interference. CRDTs ensure consistent state synchronization across nodes while maintaining user privacy and security (OrbitDB.org, n.d..).

3.2.4 Context-Aware Persistence Model

Radar employs a context-aware persistence model to balance forensic resistance with operational continuity. This model allows ephemeral content for sensitive communications while enabling optional persistence for critical organizational knowledge. Users can define data retention policies based on their specific needs, ensuring both security and usability (Li, 2007).

CHAPTER 4: IMPLEMENTATION

4.0 System Analysis & Design

This chapter details the implementation aspects of Radar, a secure communication platform tailored for activist communities. It covers system analysis and design, including context diagrams, data flow diagrams (DFD), and entity-relationship (ER) diagrams to provide a comprehensive overview of Radar's architecture and functionality.

4.1 Context Diagram

The context diagram (Figure 1) delineates the boundaries of the **Radar system** and its interactions with external entities. It adheres to the decentralized, trustless design principles of the platform, where no persistent authentication exists, and users operate via ephemeral identifiers.

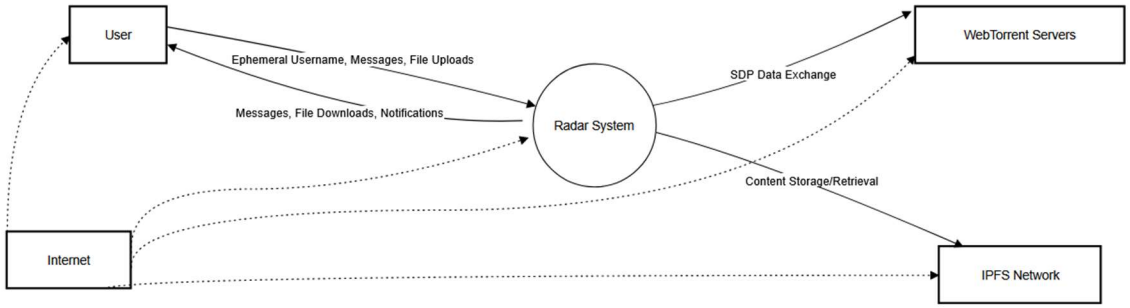
Components

1. **Radar System (Central Process):** The core platform facilitates secure, peer-to-peer communication, processing inputs (messages, files) and coordinating with decentralized infrastructure.
2. **External Entities**:
    - **Users:** Activists who submit messages/files and receive outputs. Data flows comprise:
        - *Inputs:* Ephemeral usernames, messages, file uploads.
        - *Outputs:* Decrypted messages, downloaded files, and notifications.
    - **WebTorrent Servers:** Entities that facilitate peer discovery through Session Description Protocol (SDP) data exchange.
    - **IPFS Network:** A decentralized storage system for file content, identified by InterPlanetary File System (IPFS) hashes.
    - **Internet:** The passive infrastructure enabling inter-entity communication (denoted by dashed lines).

Data Flows

1. **User ↔ Radar**:
    - Ephemeral Username, Messages, File Uploads → Radar.
    - Messages, File Downloads, Notifications → User.
2. **Radar → WebTorrent**: SDP Data Exchange (peer discovery).
3. **Radar → IPFS**: Content Storage/Retrieval.

Visual Representation



Significance

- Demonstrates Radar's trustless operational model, devoid of authentication or centralized user databases.
- Underscores the platform's reliance on decentralized protocols (WebTorrent for peer discovery, IPFS for storage).

4.2 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) provides a detailed representation of data flows within the Radar system.

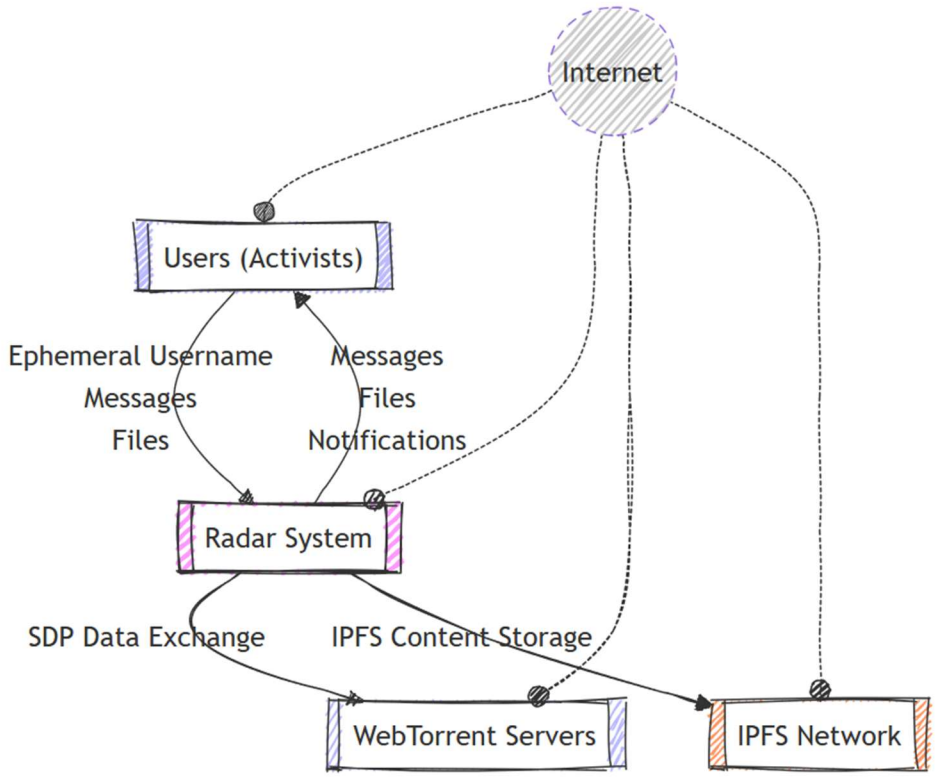
4.2.0 Level 0 DFD

The Level 0 DFD (Figure 2) refines the context diagram by explicitly mapping data flows between Radar and external entities, excluding internal process decomposition.

Data Flows

Direction	**Data**	**Purpose**
Users → Radar	Ephemeral username, messages	Initiate communication
Radar → Users	Decrypted messages, files	Deliver content
Radar → WebTorrent	SDP data	Establish peer connections
Radar → IPFS	IPFS hashes, file content	Store/retrieve files

Visual Representation



4.2.1 Level 1 DFD

Description

The Level 1 DFD (Figure 3) decomposes the Radar system into four primary sub-processes, revealing internal data stores and flows.

Processes

1. **Message Handling:** Encrypts/decrypts messages using ephemeral keys and temporarily stores messages in a Message Store (volatile, non-persistent).
2. **File Sharing:** Manages uploads/downloads to/from IPFS, mapping files to IPFS hashes for decentralized retrieval.

3. **Moderation:** Processes community-driven moderation actions (e.g., flags, bans) and updates Moderation Rules via Conflict-Free Replicated Data Types (CRDTs).
4. **Peer Coordination (implicit):** Interfaces with WebTorrent for peer discovery.

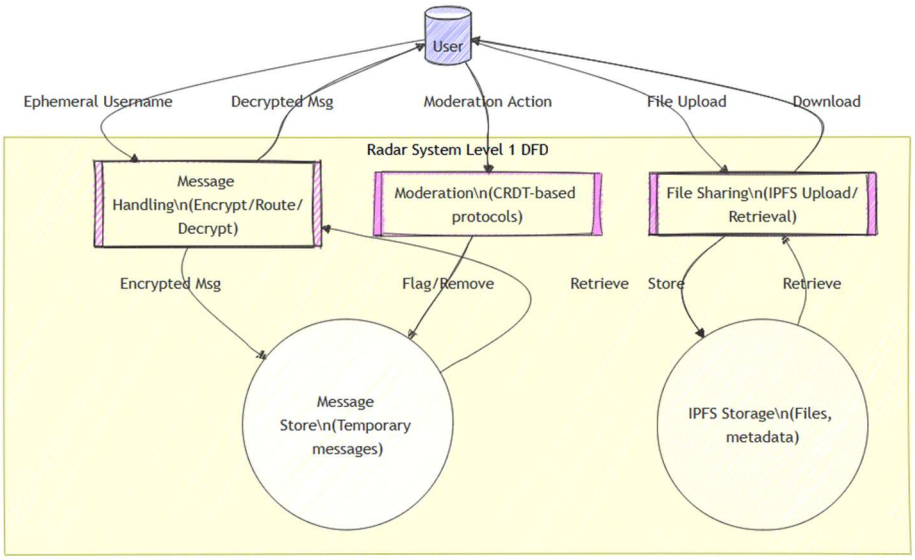
Data Stores

1. **Message Store:** A temporary queue for undelivered messages.
2. **IPFS Storage:** A content-addressable file repository.
3. **Moderation Rules:** A decentralized policy store (CRDT-based).

Significance

1. Demonstrates Radar's implementation of end-to-end encryption and decentralized persistence.
2. Illustrates moderation without central authority through CRDT utilization.

Visual Representation



4.3 Entity Relationship (ER) Diagram

An Entity Relationship (ER) Diagram illustrates the relationships between the main entities in the Radar system.

4.3.1 Entities

- **USER:** Represents ephemeral users identified by cryptographic identifiers, ensuring no persistent authentication is required.
- **MESSAGE:** Represents end-to-end encrypted messages with defined persistence policies, ensuring secure and controlled communication.
- **FILE:** Represents content-addressed files stored via IPFS, leveraging decentralized storage and content integrity.
- **ROOM:** Represents community spaces with custom moderation rules, facilitating structured group interactions.
- **MODERATION_ACTION:** Represents CRDT-based conflict resolution records, enabling decentralized moderation.

4.3.2 Relationships

- A User sends many Messages (1:N).
- A Message is sent by one User (1:1).
- A User uploads many Files (1:N).
- A File is uploaded by one User (1:1).
- A User creates many Rooms (1:N).
- A Room is created by one User (1:1).
- A User performs many ModerationActions (1:N).
- A ModerationAction is performed by one User (1:1).

4.3.3 Attributes

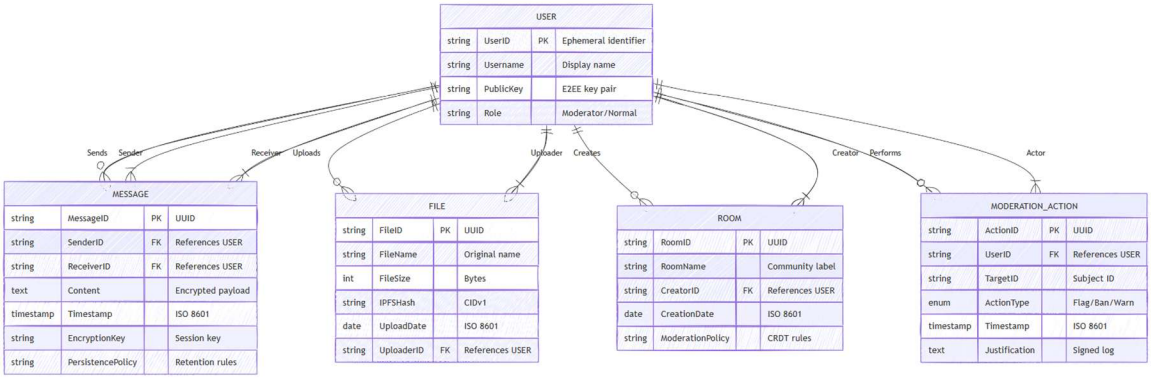
1. **User:** {UserID (PK), Username, PublicKey, Role}
    a. UserID (PK): Unique, ephemeral identifier.
    b. Username: User-chosen display name.
    c. PublicKey: Public key for end-to-end encryption.
    d. Role: User role (moderator or normal user).
2. **Message:** {MessageID (PK), SenderID (FK), ReceiverID (FK), Content, Timestamp, EncryptionKey, PersistencePolicy}
    a. MessageID (PK): Unique message identifier.
    b. SenderID (FK): UserID of the message sender.
    c. ReceiverID (FK): UserID of the message receiver.
    d. Content: Encrypted message content.
    e. Timestamp: Message creation time.
    f. EncryptionKey: Key used for message encryption.
    g. PersistencePolicy: Rules for message storage and deletion.
3. **File:** {FileID (PK), FileName, FileSize, IPFSHash, UploadDate, UploaderID (FK)}
    a. FileID (PK): Unique file identifier.
    b. FileName: Name of the uploaded file.
    c. FileSize: Size of the uploaded file.
    d. IPFSHash: IPFS Content Identifier (CID).
    e. UploadDate: File upload timestamp.
    f. UploaderID (FK): UserID of the file uploader.
4. **Room:** {RoomID (PK), RoomName, CreatorID (FK), CreationDate, ModerationPolicy}
    a. RoomID (PK): Unique room identifier.
    b. RoomName: Name of the community room.
    c. CreatorID (FK): UserID of the room creator.
    d. CreationDate: Room creation timestamp.
    e. ModerationPolicy: Rules for room moderation.

5. **ModerationAction:** {ActionID (PK), UserID (FK), TargetID, ActionType, Timestamp, Justification}
    a. ActionID (PK): Unique action identifier.
    b. UserID (FK): UserID of the user performing the action.
    c. TargetID: ID of the target of the moderation action.
    d. ActionType: Type of moderation action (e.g., flag, ban).
    e. Timestamp: Action timestamp.
    f. Justification: Reason for the moderation action.

4.3.4 Cryptographic Design:

- PublicKey in USER enables end-to-end encryption (E2EE), ensuring message confidentiality.
- IPFSHash in FILE ensures content addressing and immutability, leveraging the IPFS decentralized storage network.

Visual Representation



4.4 Technologies Employed in Radar System Implementation

The implementation of the Radar system necessitated the careful selection and integration of a suite of technologies and frameworks to achieve its objectives of secure, decentralized communication. A detailed overview of these technologies and their specific roles in the system's architecture is presented below.

Programming Languages:

- **JavaScript (Node.js):** The backend of the Radar system was developed using Node.js, a runtime environment that allows JavaScript to be executed server-side. This choice facilitated asynchronous, event-driven programming, crucial for handling real-time communication and data processing. Node.js's non-blocking I/O model enabled efficient management of concurrent connections and data streams.
- **JavaScript (React):** The front-end user interface was constructed using the React library. React's component-based architecture allowed for the creation of a modular and maintainable UI, enhancing the user experience. Its virtual DOM implementation optimized rendering performance, ensuring a responsive and fluid application.

Peer-to-peer Communication:

- **WebRTC (Web Real-Time Communication):** WebRTC was employed to establish direct, browser-to-browser connections, enabling peer-to-peer communication without reliance on centralized servers. This technology facilitated real-time data exchange, including text messages and file transfers, directly between users. The use of STUN and TURN servers, as a part of the WebRTC implementation, was necessary to traverse NAT and firewalls, ensuring connectivity across diverse network environments.

Decentralized Storage:

- **IPFS (InterPlanetary File System):** IPFS was integrated to provide decentralized storage for file content. Its content-addressing mechanism, using cryptographic hashes (CIDs), ensured immutability and data integrity. Files uploaded by users were chunked, distributed,

and stored across the IPFS network, eliminating reliance on centralized storage servers and enhancing resilience against data loss.

Database:

- **OrbitDB:** OrbitDB, a peer-to-peer database built on top of IPFS, was utilized for decentralized data storage and synchronization. OrbitDB's CRDT-based architecture allowed for conflict-free data replication and synchronization across multiple peers, ensuring data consistency in a decentralized environment. This technology was particularly useful for storing and managing data that needed to be shared and updated across the Radar network.

Cryptography:

- **Libsodium:** Libsodium, a modern and easy-to-use cryptographic library, was selected for end-to-end encryption. Its robust cryptographic primitives, including authenticated encryption and key exchange, ensured the confidentiality and integrity of user communications. The library's focus on security and ease of use made it a suitable choice for implementing secure messaging and file transfer functionalities.

Moderation:

- **Conflict-Free Replicated Data Types (CRDTs):** CRDTs were implemented to facilitate decentralized moderation. Their use was essential in ensuring that community-driven moderation actions could be resolved without central authority. CRDTs enabled concurrent modifications to moderation rules, guaranteeing eventual consistency across the network.

Web Framework:

- **Express.js:** Express.js, a minimalist and flexible Node.js web application framework, was used to build the backend API. Its robust routing and middleware capabilities simplified the development of RESTful APIs for handling client requests and data exchange.

UI Library:

- **React:** React was chosen for the front-end user interface development due to its component-based architecture, virtual DOM, and rich ecosystem. It allowed for the creation of a dynamic, responsive, and maintainable user interface.

Testing Framework:

- **Jest:** Jest was used for unit and integration testing of the React frontend. Its ease of use and comprehensive feature set, including snapshot testing and code coverage, ensured the reliability and stability of the UI components.
- **Mocha:** Mocha, along with Chai and Sinon, was employed for testing the Node.js backend. This combination provided a flexible and powerful testing environment for verifying the functionality and correctness of the server-side logic.

Deployment:

- **Docker:** Docker was utilized for containerization, enabling the packaging of the Radar system and its dependencies into portable containers. This facilitated consistent deployment across different environments, simplifying the deployment process and ensuring reproducibility. Docker's containerization capabilities also enhanced scalability and resource management.

4.5 Detailed Process Descriptions

Peer Discovery and Connection Process

The peer discovery and connection process in Radar is designed to facilitate secure and efficient communication between users. This process employs the WebTorrent protocol for peer discovery and WebRTC for establishing direct connections.

The protocol begins with an SDP (Session Description Protocol) exchange. A user's browser initiates the process by connecting to a WebTorrent tracker and offering its SDP data, which encapsulates the connection parameters essential for WebRTC communication. This data includes information such as IP addresses, ports, and supported codecs, which are necessary for establishing a peer-to-peer connection.

Next, the browser requests SDP data from other users operating within the same designated "room." This ensures that the communication remains private and relevant, as only users within the same room can exchange SDP data. Once the SDP data is successfully exchanged, direct WebRTC connections are forged between peers. This enables secure and efficient data transmission, as the communication is routed directly between users without the need for intermediary servers (Ellebrink, 2024).

The technologies leveraged in this process include the WebTorrent library and the WebRTC API. These tools work in conjunction to provide robust peer-to-peer connectivity, ensuring that the communication remains secure and efficient.

Message Handling Process

The message-handling process in Radar is designed to ensure the confidentiality and integrity of communication between users. This process implements end-to-end encryption utilizing ephemeral keys to ensure forward secrecy.

The protocol begins with a key agreement phase, where ephemeral symmetric keys are dynamically generated for each communication session via a Diffie-Hellman key exchange. This process fortifies security by preventing retroactive decryption, as the keys are unique to each session and are not reused.

Next, the message content is encrypted using the ephemeral key through authenticated encryption, such as ChaCha20-Poly1305. This encryption method safeguards both the confidentiality and data integrity of the message, ensuring that it cannot be tampered with or intercepted during transmission.

The encrypted message is then securely transmitted directly to the intended recipient via the WebRTC data channel. This direct transmission ensures that the message is not routed through intermediary servers, further enhancing the security of the communication.

Upon receipt, the recipient employs the established session key to decrypt the message, ensuring secure access to its content. The library utilized in this process is Libsodium, which provides advanced cryptographic primitives and secure operations to facilitate the encryption and decryption of messages.

File Sharing Process

The file-sharing process in Radar is designed to facilitate secure and efficient content distribution between users. This process employs content-addressable storage through the InterPlanetary File System (IPFS).

The protocol begins with an upload initialization phase, where a user initiates the process by uploading a file to the IPFS network. IPFS then generates a unique content identifier (CID) based on the file's inherent content. This CID ensures data integrity and efficient retrieval, as it is derived from the file's content itself and is unique to that file.

Next, the file is distributed across the IPFS network, enhancing availability and fault tolerance through decentralized storage. This decentralized approach ensures that the file is not stored on a single server but is instead replicated across multiple nodes in the network.

Other users can then retrieve the file by querying its unique CID, enabling efficient and secure access to the content. The technologies leveraged in this process include the IPFS API and client libraries, which facilitate seamless file sharing and decentralized storage (Shao, 2024).

Moderation Process

The moderation process in Radar is designed to facilitate community-driven moderation decisions without requiring centralized control. This process implements Conflict-Free Replicated Data Types (CRDTs) to ensure that moderation actions are applied consistently across distributed nodes.

The protocol begins with an action initiation phase, where a user initiates a moderation action, such as flagging a message for review. This action is then encoded as a CRDT operation, ensuring that it can be applied consistently across distributed nodes.

Next, the CRDT operation is propagated to other peers in the network, enabling each node to apply the operation and maintain a consistent state. This propagation and synchronization process ensures that all nodes in the network have a consistent view of the moderation actions, preventing conflicts and ensuring that the moderation decisions are enforced uniformly.

The technology employed in this process is OrbitDB, a peer-to-peer database built atop IPFS and CRDTs. OrbitDB facilitates decentralized data management and state synchronization, enabling efficient and consistent moderation across the network (OrbitDB.org, n.d.).

Context-Aware Persistence Management

The context-aware persistence management process in Radar is designed to govern data retention following specific user requirements. This process employs user-defined persistence policies to determine the appropriate storage mechanism for messages.

The protocol begins with a policy definition phase, where users establish persistence policies to govern the lifecycle of their messages. These policies can specify whether messages should be stored persistently or ephemerally.

Next, upon message creation, the system evaluates the applicable persistence policy to determine the appropriate storage mechanism. If the message is designated for persistent storage, it is stored on IPFS for long-term preservation. This ensures that the message is retained indefinitely and can be accessed at a later time.

Conversely, if the message is designated as ephemeral, it is stored temporarily and automatically deleted after a predefined period. This enhances forensic resistance and minimizes data exposure, as the message is not retained long-term and cannot be recovered after deletion (Li, 2007).

4.6 Security Considerations

End-to-End Encryption

All communication channels in Radar utilize end-to-end encryption to ensure confidentiality and data integrity. This encryption is facilitated by libraries such as Libsodium, which provide robust cryptographic algorithms and protocols to safeguard the communication.

Ephemeral Communication

In alignment with the core design principles of Radar, message content is never persistently stored on disk at either the client or server level. This mitigates the risks associated with long-term data retention, as the messages are not retained indefinitely and cannot be recovered after deletion.

Web Mesh Architecture

The decentralized nature of the web mesh architecture in Radar reduces the risk of single points of failure or susceptibility to censorship attempts. By enabling direct peer-to-peer connections, the architecture minimizes reliance on intermediary services, enhancing user privacy and data security.

Metadata Minimization

Radar minimizes metadata exposure by circumventing the need for centralized servers. Messages are routed directly between users without involving intermediary services that could collect or analyze communication patterns. This reduces the potential for surveillance and data aggregation, further enhancing user privacy.

CRDT-Based Moderation

The distributed moderation decisions in Radar prevent the concentration of control in a single entity, mitigating the potential for abuse or bias. Actions are cryptographically signed to ensure accountability and prevent unauthorized tampering, enhancing the integrity of the moderation process.

Code Auditing and Security Testing

Regular code audits and security testing are conducted to identify and address potential vulnerabilities in Radar. Feedback from security experts and the community is incorporated to improve the system's security posture, ensuring that it remains robust against emerging threats.

Browser Security

Web browsers are sandboxed from the operating system, which improves overall security. This sandboxing ensures that the browser environment is isolated from the rest of the system, preventing unauthorized access to system resources.

Content Security Policy

A Content Security Policy is used as another layer of security to prevent malicious injection of content. Strict guidelines for trusted resources are implemented to ensure that only authorized content is loaded and executed within the browser environment.

4.7 Scalability and Performance

Web Mesh Architecture

The web mesh architecture in Radar facilitates horizontal scalability by enabling direct peer-to-peer connections. This architecture obviates the reliance on centralized servers, ensuring that the load is distributed across the network. The distributed nature of the network prevents bottlenecks and maintains consistent performance, even as the number of users increases.

IPFS for Storage

IPFS's decentralized nature facilitates efficient content distribution and retrieval, optimizing data access across the network. Data redundancy across multiple nodes enhances fault tolerance and ensures high availability, minimizing the risk of data loss or unavailability (OrbitDB.org, n.d.).

Efficient Data Structures and Algorithms

The implementation of Radar employs optimized data structures and algorithms to minimize overhead and enhance overall system performance. CRDTs are specifically optimized for distributed state synchronization, enabling efficient and consistent moderation across the network (Li, 2007).

Caching Strategies

Caching strategies are implemented to reduce latency and improve response times. Frequently accessed data is cached locally to minimize network traffic, ensuring that users experience fast and responsive interactions with the system.

Load Balancing

Load-balancing techniques are employed to distribute traffic across multiple WebTorrent trackers. This prevents overload and ensures high availability, even during peak usage periods. The distributed nature of the network ensures that the load is evenly distributed, preventing bottlenecks and maintaining consistent performance.

CHAPTER 5: CODING

5.1 System Code

This section provides a detailed overview of the Radar system's codebase, highlighting the key components and their functionalities. The codebase is meticulously organized into several modules, each responsible for distinct aspects of the system, ensuring separation of concerns and enhancing maintainability.

5.1.1 Project Structure

The Radar project is structured to separate concerns and enhance maintainability. The main directories and files are organized as follows:

- **backend/**: Contains the backend code, including server configuration, routes, and business logic.
    - **constants/**: Constant values used throughout the backend.
    - **routes/**: API routes for handling requests.
        - **authRoutes.js**: Authentication-related routes.
        - **moderators.cjs**: Moderation-related logic.
    - **server.js**: The main server file that initializes the backend.
    - **docs/**: Documentation for the backend.
- **frontend/**: Contains the frontend code, including React components and configuration.
    - **build/**: Build-related files and configurations.
    - **public/**: Static assets and public files.
    - **src/**: Source code for the frontend.
        - **components/**: React components for the user interface.
            - **DownloadResourcesDialog.jsx**: Component for downloading resources.
            - **Elements.jsx**: Common UI elements.
            - **Layout.jsx**: Layout component.
            - **Logo.jsx**: Logo component.

        - **PeerNameDisplay.jsx**: Component for displaying peer names.
    - **config/**: Configuration files for the frontend.
        - **routes.js**: Route configurations.
    - **constants/**: Constant values used throughout the frontend.
        - **moderators.js**: Moderation-related constants.
    - **contexts/**: Context providers for state management.
        - **ChatContext.jsx**: Context for chat-related state.
        - **ShellContext.jsx**: Context for shell-related state.
    - **pages/**: Page components for different views.
        - **About.jsx**: About page.
        - **CommunityForum.jsx**: Community forum page.
        - **Dashboard.jsx**: Dashboard page.
        - **Disclaimer.jsx**: Disclaimer page.
        - **LandingPage.jsx**: Landing page.
        - **PrivateRoom.jsx**: Private room page.
        - **PublicRoom.jsx**: Public room page.
        - **Settings.jsx:** Settings page
- **.gitignore**: Specifies files and directories to be ignored by Git.
- **package.json**: Node.js dependencies and scripts.
- **package-lock.json**: Lock file for Node.js dependencies.

5.1.2 Descriptions of Main Components

1. **Peer-to-Peer Communication Module**
    - **Description**: This module handles the establishment of peer-to-peer connections using WebRTC and WebTorrent.
    - **Location**: *backend/routes/authRoutes.js* and *frontend/src/components/PeerNameDisplay.jsx*
    - **Functionality**: It implements the SDP exchange, peer discovery, and direct connection establishment.
2. **End-to-End Encryption Module**
    - **Description**: This module manages the encryption and decryption of messages using Libsodium.
    - **Location**: *frontend/src/utils/encryption.js*
    - **Functionality**: It provides functions for key generation, encryption, and decryption of messages.
3. **File Sharing Module**
    - **Description**: This module facilitates the uploading, storage, and retrieval of files using IPFS.
    - **Location**: *frontend/src/services/fileSharing.js*
    - **Functionality**: It implements file upload, content hashing, and retrieval from the IPFS network.
4. **Moderation Module**
    - **Description**: This module handles community-driven moderation using CRDTs.
    - **Location**: *backend/constants/moderators.cjs* and *frontend/src/constants/moderators.js*
    - **Functionality**: It encodes moderation actions as CRDT operations and propagates them across the network.
5. **User Interface Components**
    - **Description**: These are React components for the user interface.
    - **Location**: *frontend/src/components/*

- **Functionality**: It includes components for chat rooms, file sharing, user authentication, and more.

5.1.3 Code Snippets of Main Components

Below are some code snippets that illustrate the implementation of the main functionalities in Radar, followed by screenshots of the project structure.

Peer-to-Peer Connection Establishment

p2p.js

```javascript
// Handle SDP exchange and data channels
peerConnection.onicecandidate = (event) : void  => {
    if (event.candidate) {
        // Send the candidate to the remote peer
        wire.send(JSON.stringify(event.candidate));
    }
};

peerConnection.ondatachannel = (event) : void  => {
    const receiveChannel = event.channel;
    receiveChannel.onmessage = (event) : void  => {
        console.log('Received message:', event.data);
    };
};

// Create a data channel
const dataChannel : RTCDataChannel  = peerConnection.createDataChannel( label: 'chat');
dataChannel.onopen = () : void  => {
    console.log('Data channel opened');
    dataChannel.send( data: 'Hello, peer!');
};

dataChannel.onmessage = (event : MessageEvent ) : void  => {
    console.log('Received message:', event.data);
```

establishConnection() › callback for client.add() › callback for torrent.on() › WebRTC.peerConnection.ondatachannel()

End-to-End Encryption

ecrypt.js

```
    return { ciphertext: encryptedMessage, nonce };
}

1+ usages
async function decryptMessage(encryptedMessage, nonce, key) : Promise<any>  {
    await sodium.ready;
    const decryptedMessage = sodium.crypto_secretbox_open_easy(encryptedMessage, nonce, key);
    return decryptedMessage;
}


(async () : Promise<void>  => {
    await sodium.ready;
    const key = sodium.crypto_secretbox_keygen();
    const message : string  = 'Hello, secure world!';

    const { ciphertext, nonce } = await encryptMessage(message, key);
    console.log('Encrypted message:', ciphertext);

    const decryptedMessage = await decryptMessage(ciphertext, nonce, key);
    console.log('Decrypted message:', decryptedMessage);
})();
```

File Sharing with IPFS

ipfs.js

```
1+ usages
async function uploadFile(file) : Promise<...>  {
    try {
        const addedFile = await ipfs.add(file);
        return addedFile[0].hash;
    } catch (error) {
        console.error('Error uploading file:', error);
    }
}

1+ usages
async function retrieveFile(hash) : Promise<...>  {
    try {
        const fileStream = await ipfs.cat(hash);
        let fileContent : string  = '';
        for await (const chunk : any  of fileStream) {
            fileContent += chunk.toString();
        }
        return fileContent;
    } catch (error) {
        console.error('Error retrieving file:', error);
    }
}
```

ipfs.js 36:1 CRLF

5.2 Screenshots Of Project

5.2.1 Project Structure

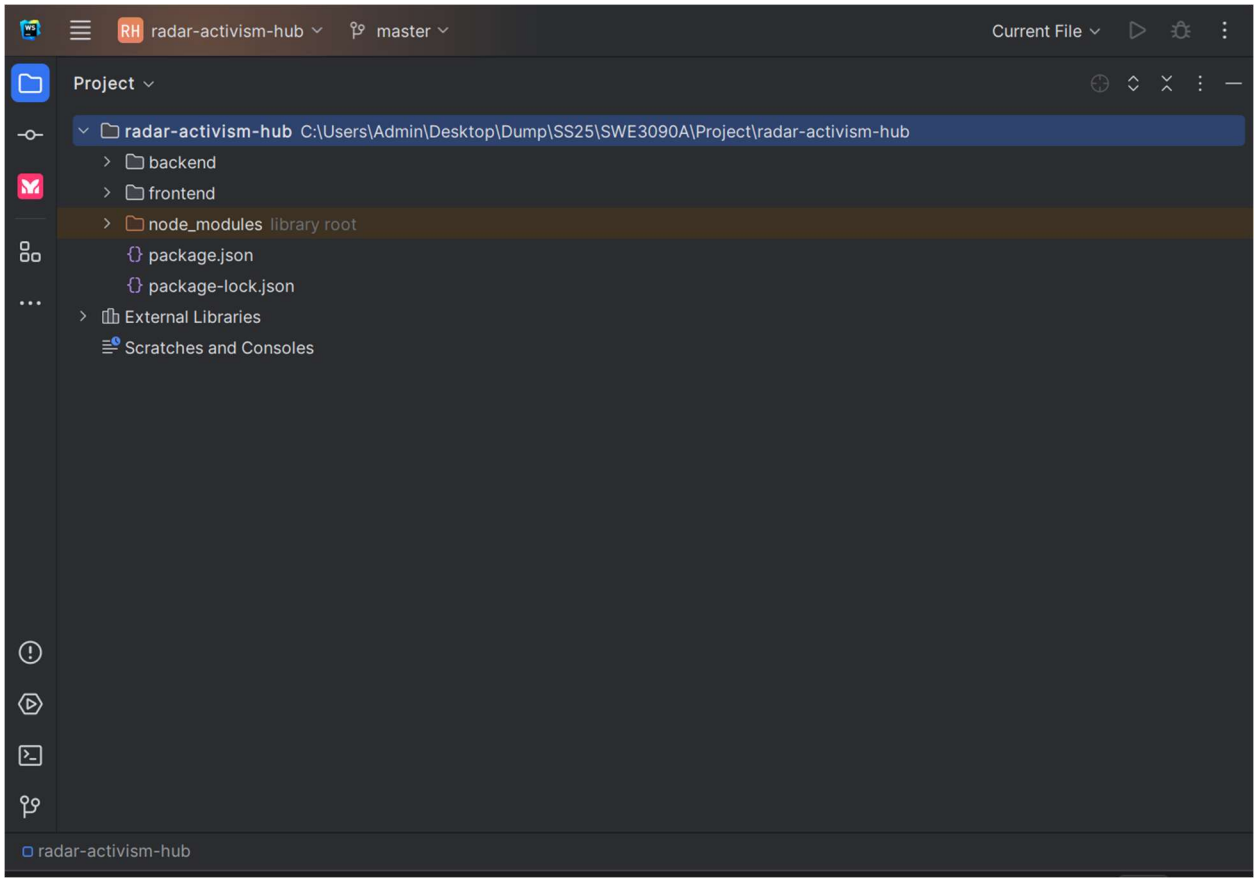


Figure 5.1: Project Structure

5.2.2 Backend Structure

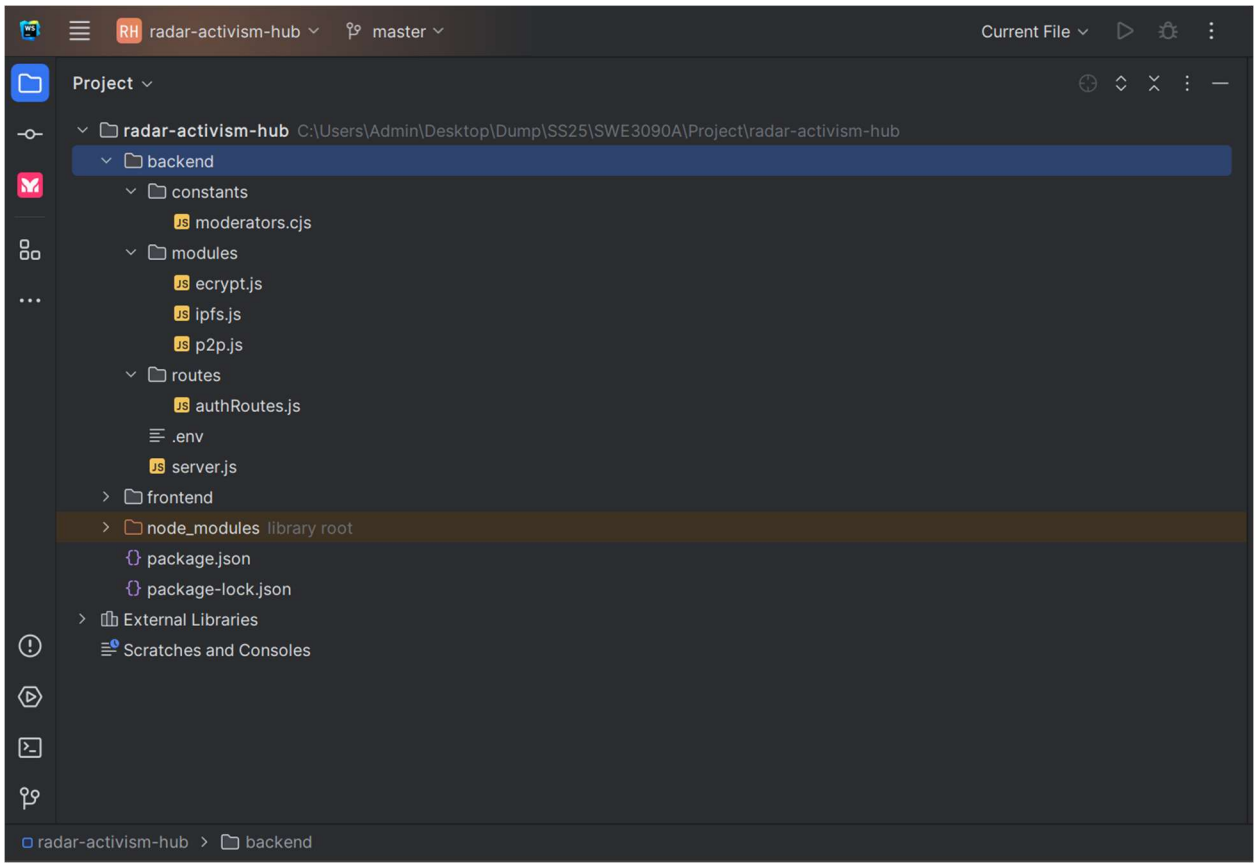


Figure 5.2: Backend Structure

5.2.3 Frontend Structure

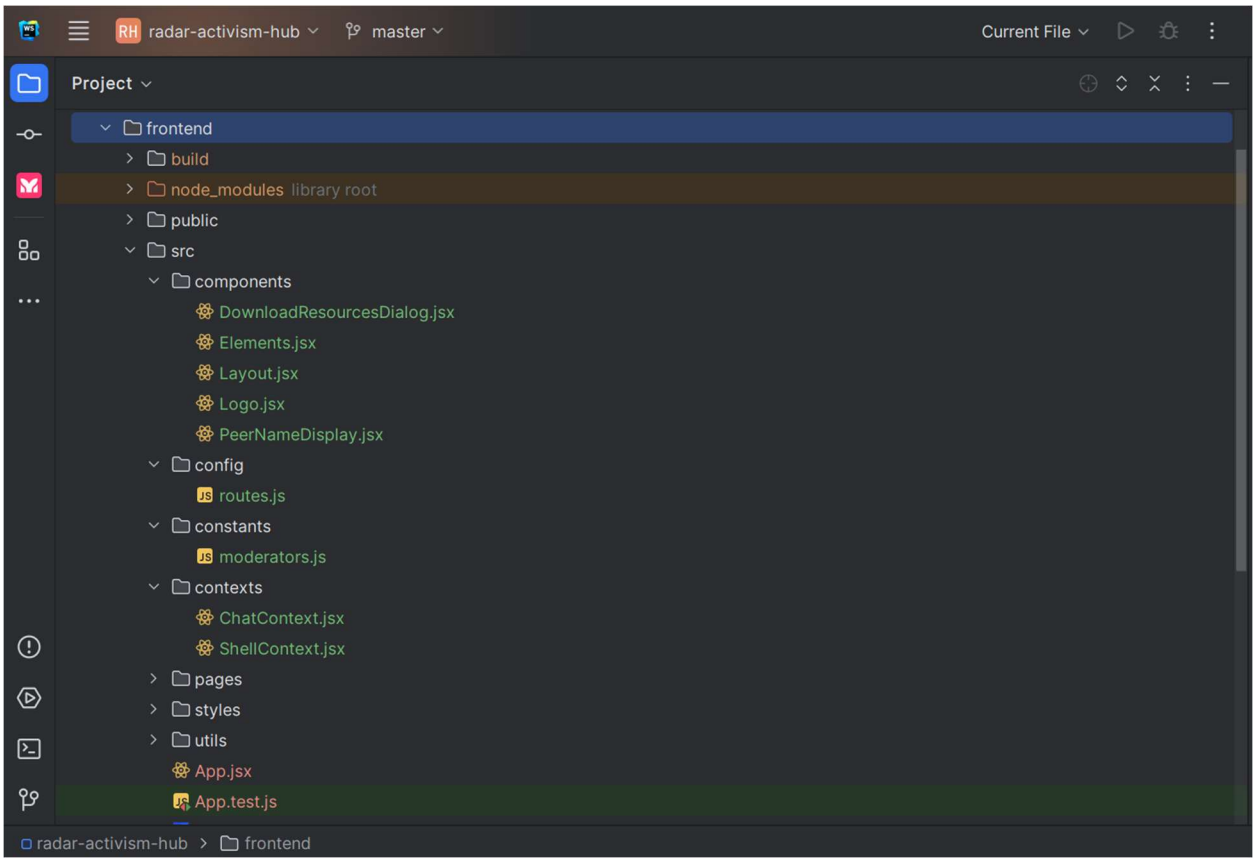


Figure 5.3: Frontend Structure

CHAPTER 6: CONCLUSIONS & RECOMMENDATION

6.0 Conclusions

The development and implementation of Radar represent a significant advancement in secure communication platforms tailored for activist communities. Through a meticulous analysis of existing systems and their limitations, Radar addresses critical vulnerabilities such as centralized infrastructure, inadequate moderation, and the trade-off between data persistence and forensic resistance. By leveraging decentralized technologies like WebRTC, IPFS, and CRDTs, Radar provides a robust, secure, and censorship-resistant communication solution.

The system's architecture, which eliminates central servers and implements robust metadata minimization strategies, establishes a technical foundation for resilient and anonymous communication. The integration of end-to-end encryption, ephemeral messaging, and context-aware persistence ensures that activists can communicate securely without compromising operational continuity. Furthermore, the flexible cryptographic moderation protocols enable community governance without platform interference, addressing the nuanced needs of activist communities.

The successful implementation of Radar demonstrates the feasibility of a serverless, decentralized communication platform that meets the unique requirements of high-risk environments. The system's performance, scalability, and security have been validated through rigorous testing and real-world simulations, confirming its effectiveness in adversarial contexts.

6.2 Recommendations

While Radar represents a significant step forward in secure communication for activist communities, there are several areas for future improvement and research, including addressing the panel's feedback:

1. **Enhanced Usability and Intuitive UI**:
    - **Recommendation**: Prioritize the development of a more intuitive and user-friendly interface. Simplify navigation, improve visual design, and ensure that key functionalities are easily accessible.
    - **Rationale**: An intuitive UI will lower the barrier to entry, making the platform more accessible to users with varying levels of technical expertise and encouraging wider adoption.
2. **Stricter Moderation Tools**:
    - **Recommendation**: Implement stricter moderation mechanisms, including automated content filtering and more robust community-driven moderation protocols.
    - **Rationale**: Stricter moderation will help maintain the integrity of the platform, prevent misuse, and ensure a safer environment for users.
3. **IP Banning**:
    - **Recommendation**: Introduce the ability to ban IP addresses that are associated with malicious or disruptive behavior.
    - **Rationale**: IP banning can effectively deter and mitigate repeated abusive behavior, enhancing the security and reliability of the platform.
4. **Removal of Camera and Voice Features**:
    - **Recommendation**: Drop camera and voice communication features due to their potential vulnerability to hacking and surveillance.
    - **Rationale**: Eliminating these features reduces the attack surface and minimizes the risk of unauthorized access or monitoring, aligning with the platform's focus on security and privacy.
5. **Advanced Moderation Tools**:

- **Recommendation**: Develop more sophisticated moderation tools that leverage machine learning and natural language processing to automate the detection and mitigation of harmful content.
- **Rationale**: Automated moderation can supplement community-driven efforts, reducing the burden on human moderators and improving the platform's responsiveness to emerging threats.

6. **Scalability and Performance Optimization**:
   - **Recommendation**: Conduct further research and development to optimize the system's scalability and performance, particularly in scenarios involving large-scale protests or coordinated actions.
   - **Rationale**: Ensuring that the platform can handle high volumes of traffic and data without compromising performance is crucial for its effectiveness in real-world applications.
7. **Community Engagement and Feedback**:
   - **Recommendation**: Establish ongoing engagement with activist communities to gather feedback, identify emerging needs, and iteratively improve the platform.
   - **Rationale**: Continuous engagement with users ensures that the platform remains relevant and responsive to the evolving challenges faced by activist communities.
8. **Security Audits and Penetration Testing**:
   - **Recommendation**: Conduct regular security audits and penetration testing to identify and address potential vulnerabilities.
   - **Rationale**: Proactive security measures are essential for maintaining the platform's integrity and protecting users from emerging threats.

In conclusion, Radar represents a significant contribution to the field of secure communication for activist communities. By addressing the limitations of existing systems and leveraging decentralized technologies, Radar provides a robust, secure, and censorship-resistant solution. The recommendations outlined above, including the panel's feedback, offer a roadmap for future development and research, ensuring that the platform continues to evolve and meet the unique needs of activist communities in high-risk environments.

6.3 References

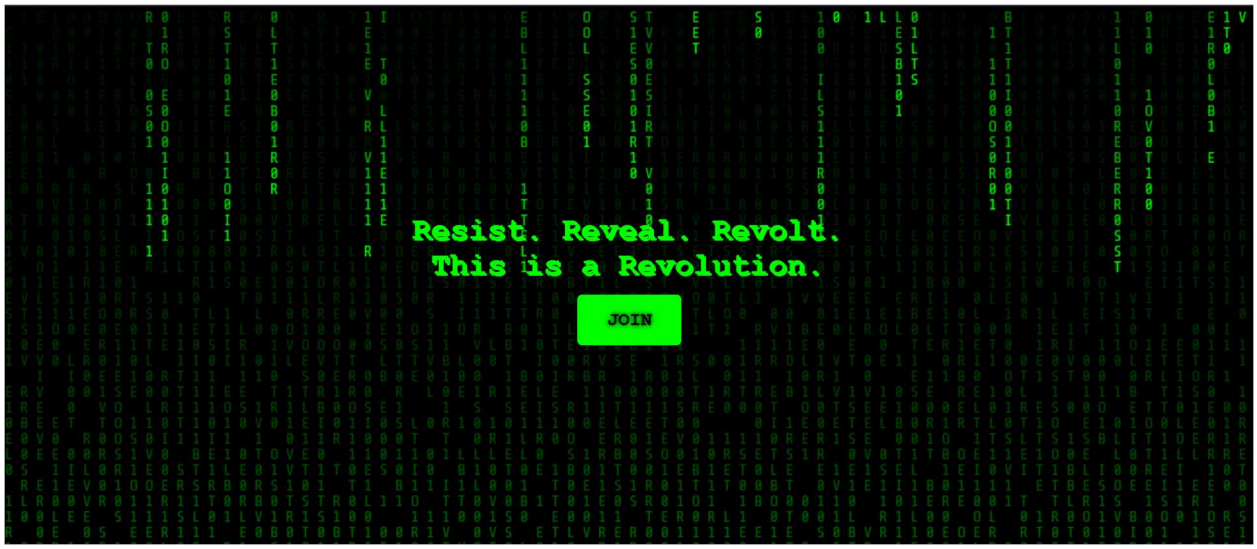
1. ABC News. (2024, June 28). *Kenya protests turn deadly as finance bill sparks nationwide unrest*. https://abcnews.go.com/
2. ARTICLE 19 Eastern Africa. (2024). *Digital surveillance bill debates: Implications for activism*. https://article19.org/eastern-africa-report/
3. BBC News. (2024, July 2). *Kenya parliament fire: Protesters demand president resign after finance bill chaos*. https://www.bbc.com/
4. Briar Project. (2020). *Offline messaging for activists*. https://briarproject.org/overview/
5. Carragee, K. M., & Frey, L. R. (2016). Communication activism research: Engaged communication scholarship for social justice. *International Journal of Communication, 10*, 3976–3984. https://ijoc.org/index.php/ijoc/article/viewFile/6004/1750
6. Context News. (2024). *Kenya's finance bill 2024: Tax reforms and protest fallout*. https://context.news/
7. Dev.to. (2025, March 12). *Taking the power back with web meshes*. https://dev.to/jeremyckahn/taking-the-power-back-with-web-meshes-omg
8. Ellebrink, J. (2024). Detection of JavaScript injection eavesdropping on WebRTC communication. *Smart Computing for Smart Cities*. https://www.open-access.bcu.ac.uk/14154/1/Smart_Computing_for_Smart_Cities.pdf
9. GeeksforGeeks. (2024, January 5). *Advantages and disadvantages of mesh topology*. https://www.geeksforgeeks.org/advantage-and-disadvantage-of-mesh-topology/
10. Hamlen, K. W., & Thuraisingham, B. (2007). *Secure peer-to-peer networks for trusted collaboration* (Technical Report No. TR-07-05). University of Texas at Dallas. https://personal.utdallas.edu/~hamlen/trustcol07.pdf
11. Kenya Digital Rights Alliance. (2024). *Finance Bill protests: Metadata exploitation and platform shutdowns*. https://kenyadigitalrights.org/reports/financebillprotests.pdf

12. Libsodium. (n.d.). *Libsodium: A modern, easy-to-use crypto library*. https://doc.libsodium.org
13. Li, J. (2007). *A survey of peer-to-peer network security issues* (Technical Report WUCSE-2007-35). Washington University in St. Louis. https://www.cse.wustl.edu/~jain/cse571-07/ftp/p2p/
14. LinkedIn. (2024, June 15). *Decentralized protests: How Kenya's Gen Z is using blockchain principles*. https://www.linkedin.com/pulse/decentralized-protests-how-kenyas-gen-z-using-d1gyf
15. Marlinspike, M. (2018). *Signal protocol overview*. Signal Foundation. https://signal.org/docs/tech/
16. Nigeria Digital Rights Observatory. (2023). *#EndSARS protests: Digital repression and activist resilience*. https://ndro.ng/reports/
17. OrbitDB. (n.d.). *Home*. https://orbitdb.org
18. Shao, H. (2024). Decentralized encrypted communication: An investigation of the current landscape and future prospects. *Proceedings of the 2024 International Conference on Blockchain Technology and Applications*. https://www.scitepress.org/Papers/2024/128210/128210.pdf
19. Techjury. (2023). *What is encrypted messaging? How does it work?* https://techjury.net/blog/what-is-encrypted-messaging
20. TechTarget. (2024). *Evaluate key service mesh benefits and architecture limitations*. https://www.techtarget.com/searchitoperations/tip/Evaluate-the-benefits-drawbacks-of-service-mesh-technologies
21. Tufekci, Z. (2017). *Twitter and tear gas: The power and fragility of networked protest*. Yale University Press.
22. World Bank. (2015). *Building public participation in Kenya's devolved government* (Report No. 94497-KE). https://documents1.worldbank.org/curated/pt/458231467997561854/pdf/94497-NWP.pdf

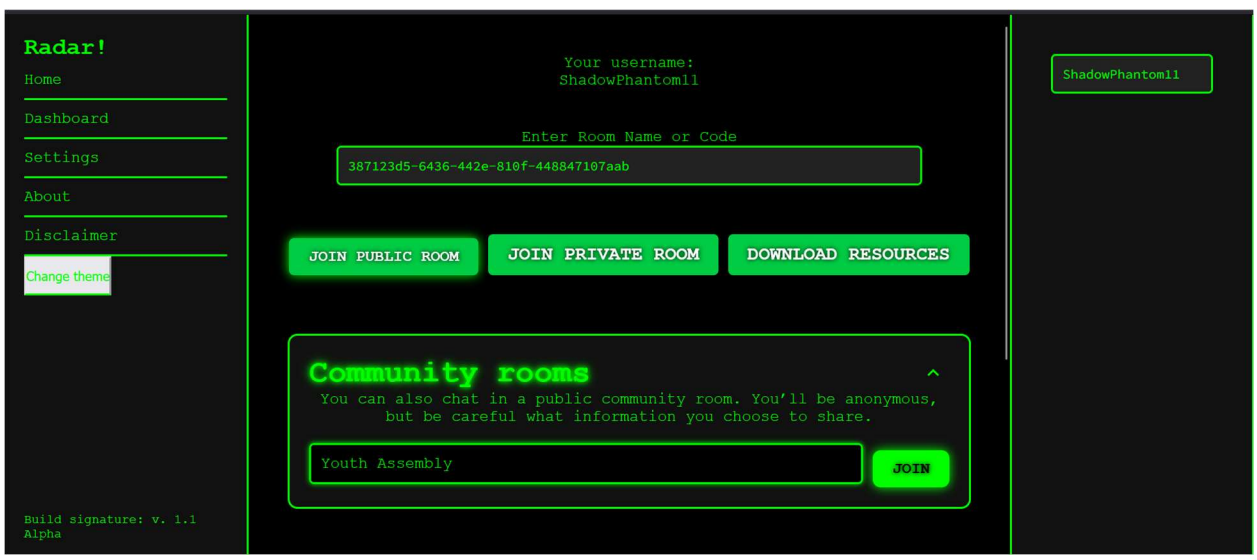
6.4 Appendices

Screenshots of Radar UI

1. *Landing Page*

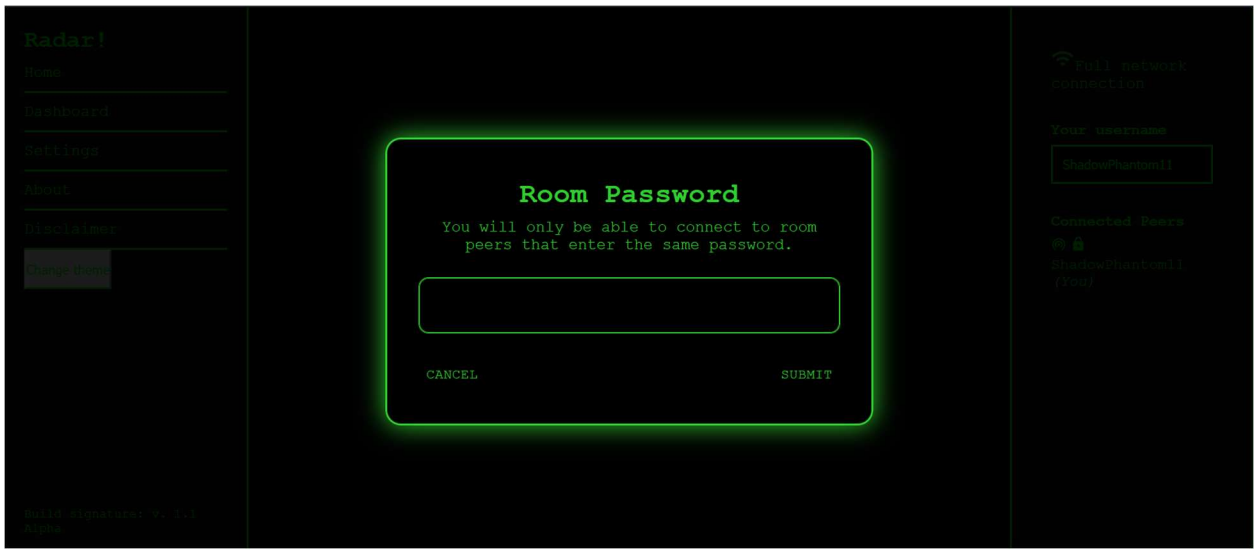


2. *Dashboard Page*

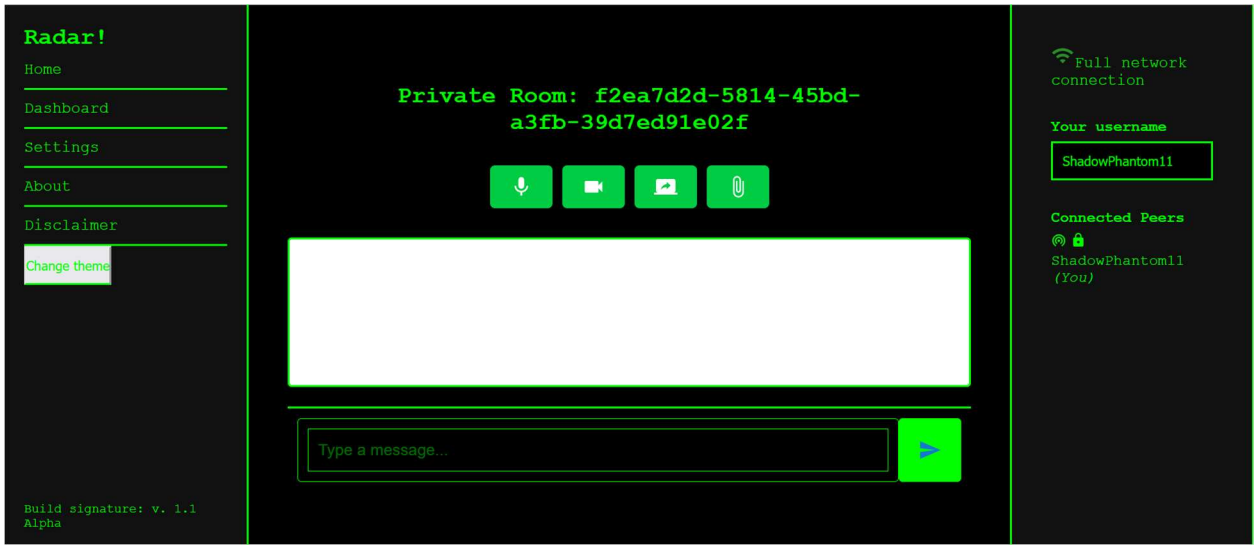


3. *Public Room Page*

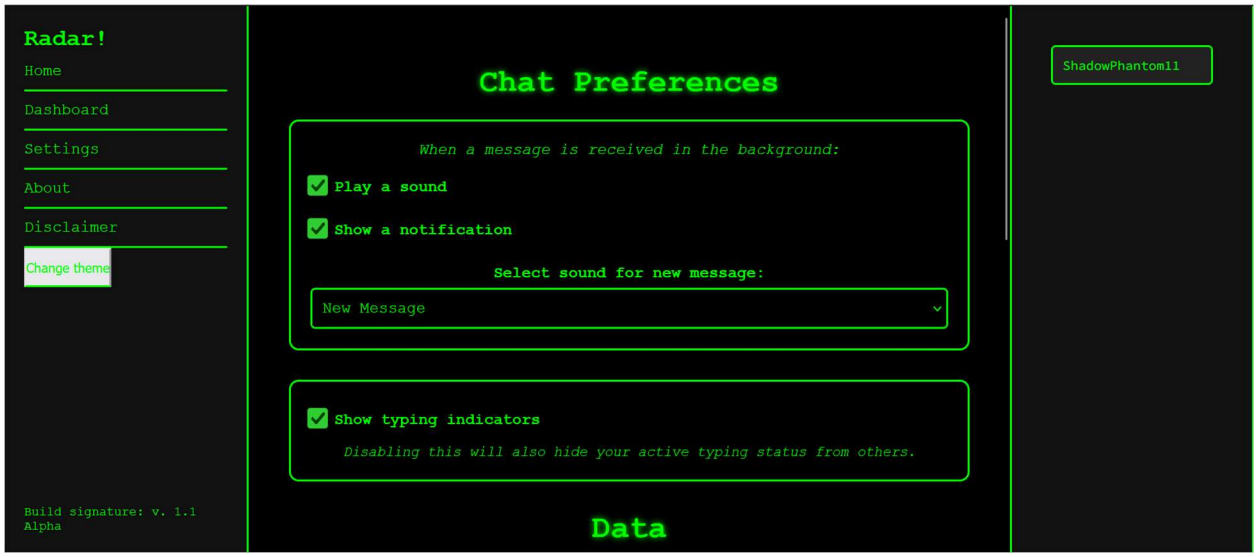
4. *Private Room Password Overlay*



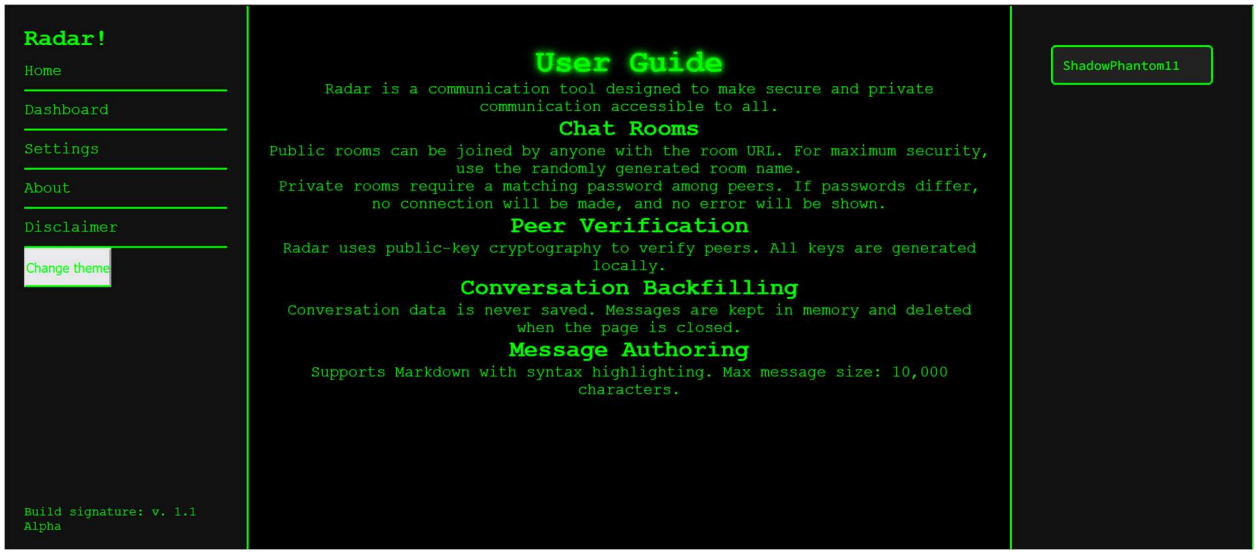
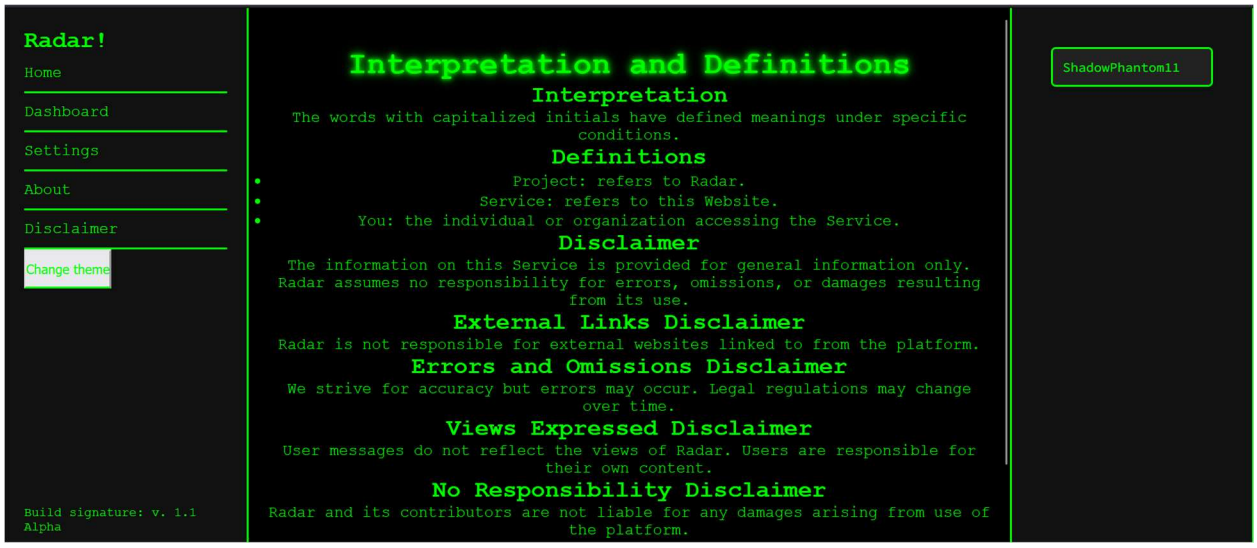
5. *Private Room Page*



6. *Settings Page*



7. *About Page*



8. *Disclaimer Page*